

Penna Simulation Summary

Natasha Kiper

September 2026

Contents

1	Introduction	1
1.1	Parameters	1
2	Results	2
2.1	Effect of varying T , R , and r_{birth}	4
3	Summary and further directions	4
4	Implementation details	6
4.1	Overview	6
4.2	Build and Project Structure	6
4.3	Domain Representation	6
4.4	Key Design Decisions	6

1 Introduction

The Penna model [1] is a bit string model to simulate population aging. It implements a population of $N(t)$ individuals at time t , each with a genome of length B . The genome is a bit string determined at birth. If the i -th bit is one, the individual will suffer a mutation at time i . An individual who has passed the mutation threshold T will stay alive with probability $1 - N(t)/N_{\text{max}}$, where N_{max} is the maximal population size. Individuals that are older than the reproductive age R generate one baby with probability r_{birth} , whose genome will be the same as the parents', except for M randomly changed bits.

1.1 Parameters

- B : genome size
- T : mutation threshold
- N_{max} : maximal number of individuals
- M : mutation rate ($M \leq B$)
- R : reproductive age
- r_{birth} : birth rate ($0 \leq r_{\text{birth}} \leq 1$)
- N_0 : initial population size
- T_{max} : maximal simulated time

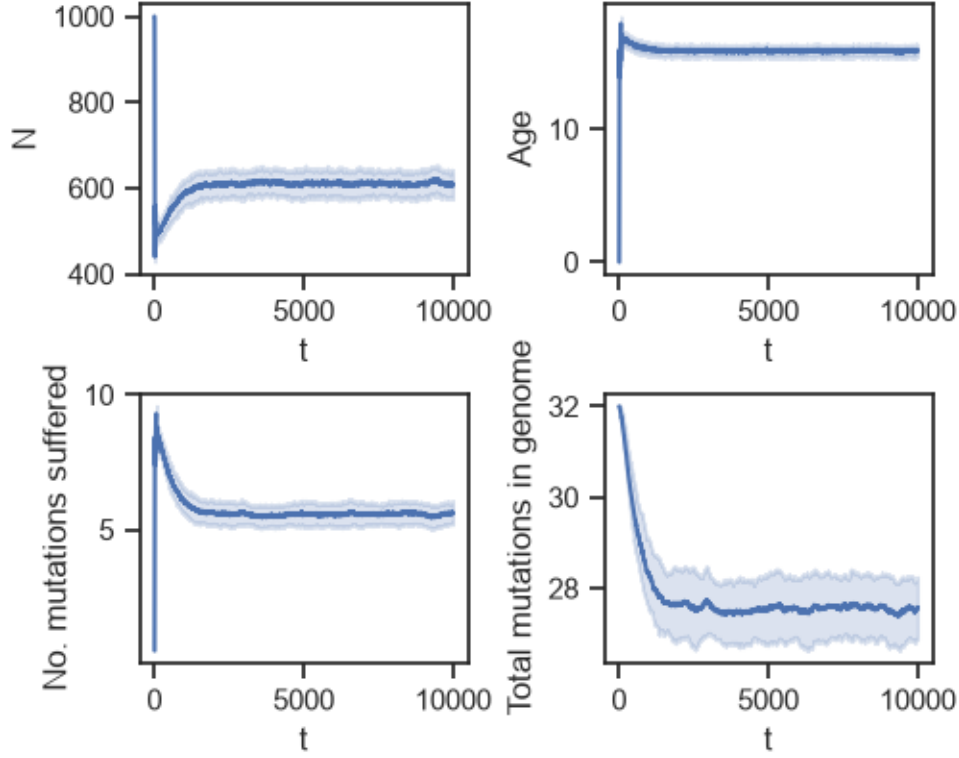


Figure 1: Population size, average age, number of suffered and total mutations as a function of time for the baseline stable population. The values shown are averages across 100 iterations of the simulation, and error bars correspond to standard deviations.

2 Results

First, a parameter set that leads to a stable population was found. The parameters used, shown in the table below, are similar to those used in Ref. [1] but adapted to the larger bitset used as the genome (64 instead of 32 bits).

B	T	R	$N_{\max}$	r_{birth}	M	N_0	$T_{\max}$
64	2	16	10^5	0.1	4	10^3	10^4

For each simulation, the population was initialized with N_0 animals of age 0 with $B/2$ bad genes at random positions.

Figure 1 shows the evolution of the population size N , the average age, the total number of mutations already suffered by animals in the population, and the total number of mutations in the genomes of all animals. All values start by changing dramatically, as animals die but cannot reproduce for $t < R$. When reproduction begins, the population evolves towards a stable state, which it reaches for $t \approx 1000$.

We can also extract the distribution of ages, suffered mutations, and total mutations in genome in the final population at $t = 10^5$, see Fig. 2. Here, the suffered mutations are the amount of mutations an animal that is still alive has already suffered during its lifetime. Note that as the population is initialized with 32 bad genes per animals and the mutation rate is $M = 4$, only even numbers of total mutations are possible, explaining the gaps in the distribution of total mutations.

The age distribution has a truncated shape, with lower ages being more prevalent than older ones. The number of mutations suffered is peaked at 1 and then rapidly decays towards larger values, as expected for a mutation threshold $T = 2$. The total number of mutations is normally distributed around a value of ≈ 28 .

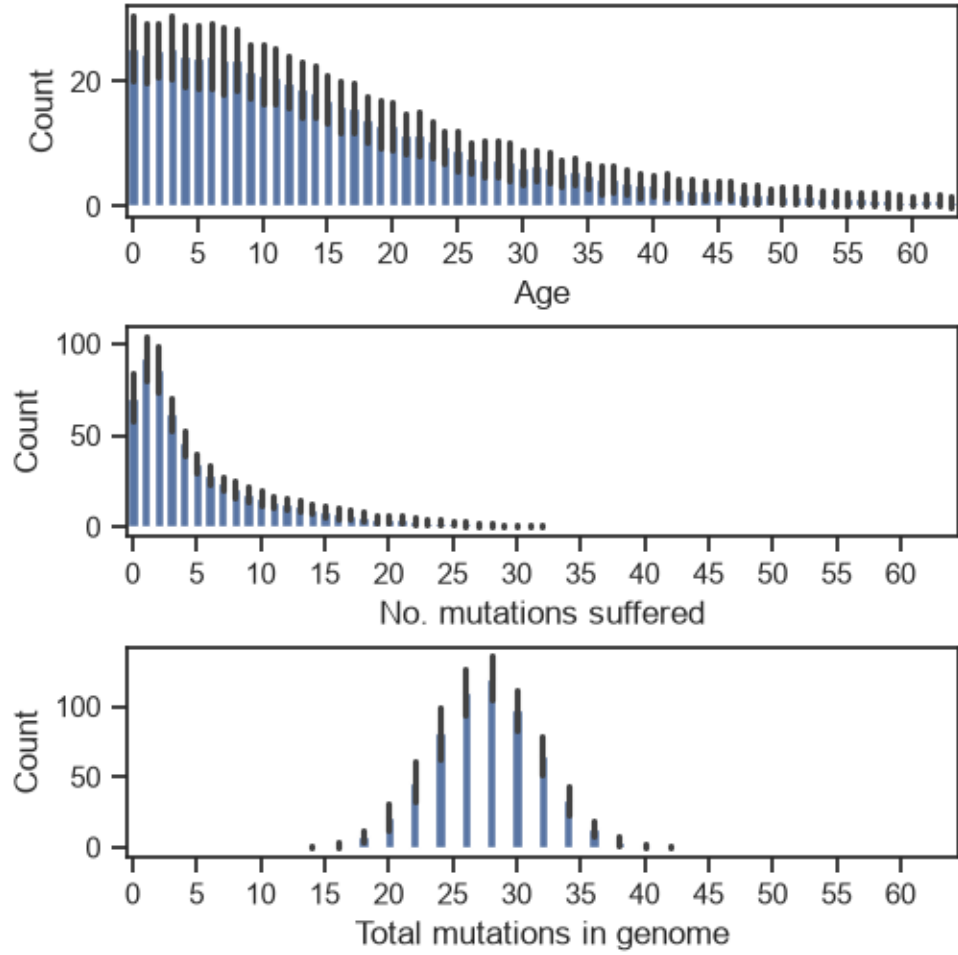


Figure 2: Distribution of ages, already suffered mutations, and total mutations in the final population at $t = 10^5$.

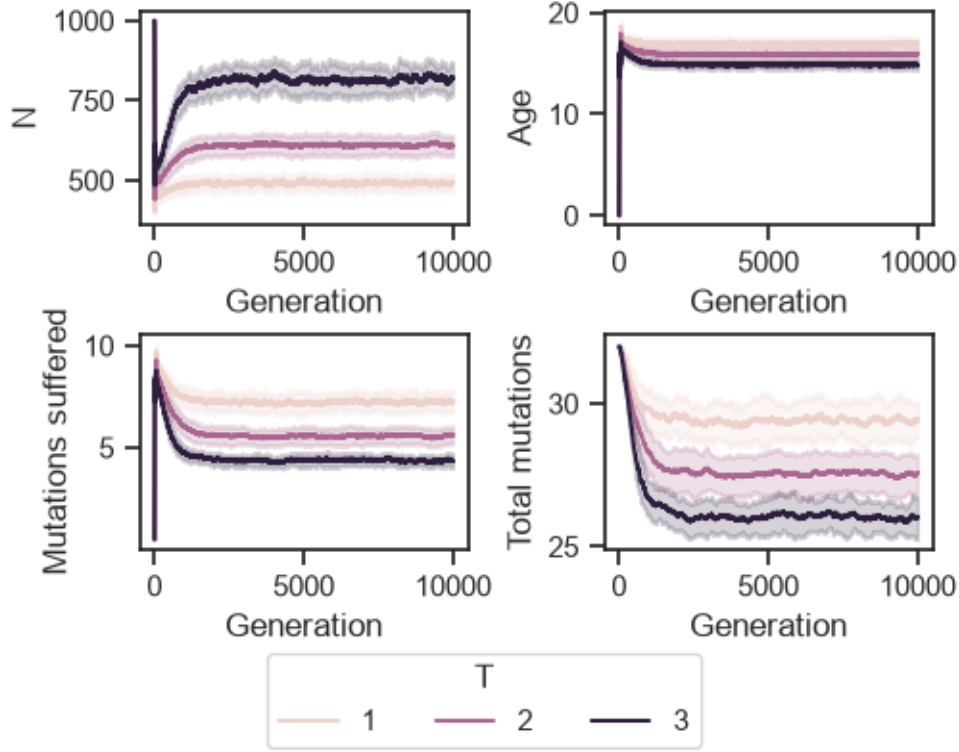


Figure 3: Effect of mutation threshold T on population evolution.

2.1 Effect of varying T , R , and r_{birth}

Next, we study the effect of varying the mutation threshold T , the reproductive age R , and the birth rate r_{birth} on the population's evolution. To this end, we vary each parameter to one higher and one lower value than the base case presented above, and run the simulation for 40 iterations each.

We start with the mutation threshold T , where we study the values $T = 1, 3$ in addition to the base case $T = 2$. The population evolutions can be seen in Fig. 3. We see that for higher (lower) T , the population stabilizes at a higher number N . This is expected, as a higher threshold allows more animals to live longer. It is somewhat surprising that the average age, number of mutations suffered, and total mutations suffered stabilize at a value that is inversely proportional to the mutation threshold. Naively, one would perhaps expect a lower mutation threshold to lead to increased evolutionary pressure and therefore a trend toward fewer mutations in the genome and a younger population. However, the probability of an animal to survive is not only determined by its mutations, but also the factor $1 - N/N_{\text{max}}$, which simulates population capping due to resource scarcity. The larger population N in the $T = 3$ case thus leads to an increased probability of death. Note that the number of mutations only affect survival probability in a binary way, additional mutations suffered beyond the threshold do not increase death probability further.

A similar effect is observable when changing the reproductive age R (Fig. 4) and the birth rate r_{birth} (Fig. 5). A lower reproductive age and larger birth rate both lead to larger populations, increasing evolutionary pressure.

3 Summary and further directions

We implemented the Penna biological aging model in C++ and found a set of parameters that leads to a stable population. The effect of varying some parameters on the population evolution was studied. We found that the primary effect was on the number of animals in the stable population, which in turn affected evolutionary pressure due to the population capping mechanism in the model.

A possible future extension is to include fishing in the model, as described in Ref. [2].

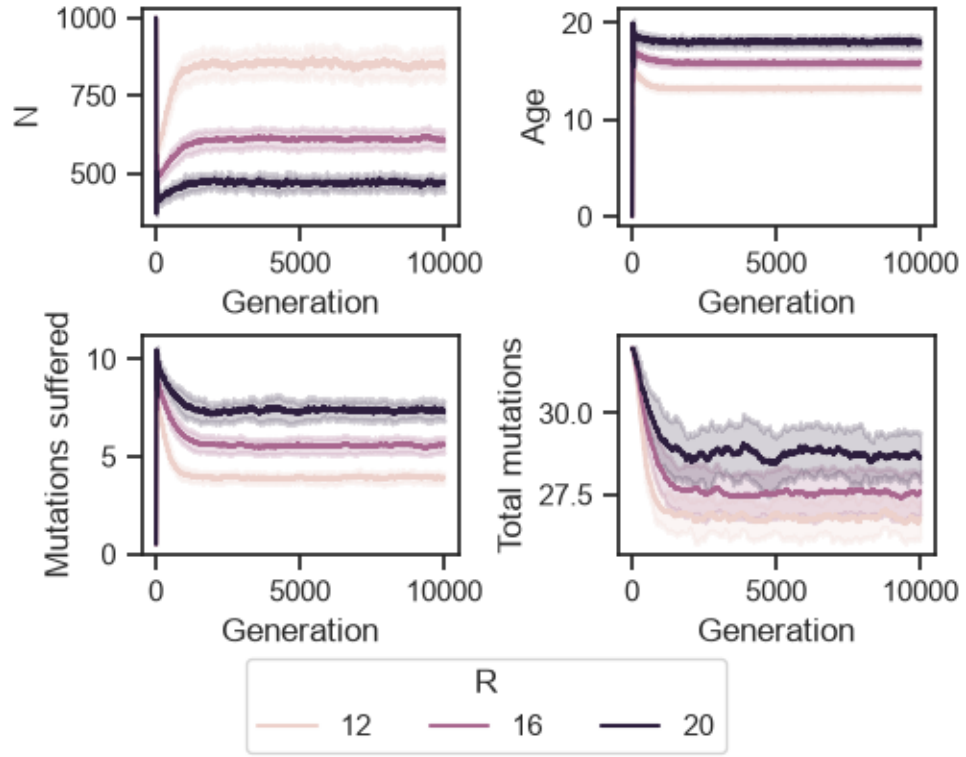


Figure 4: Effect of reproductive age R on population evolution.

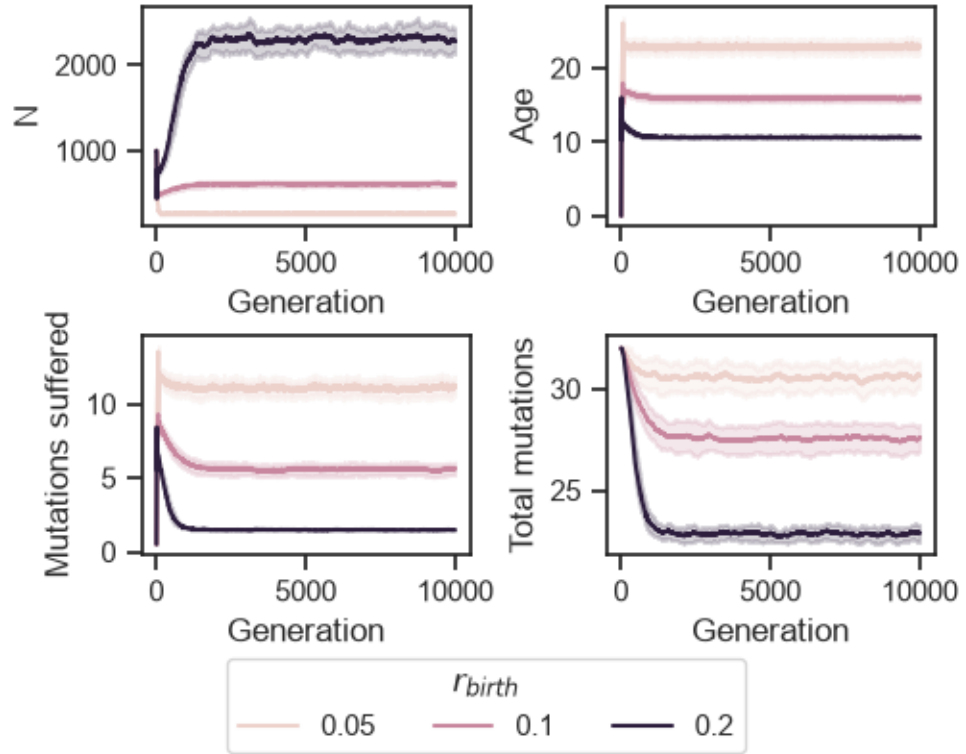


Figure 5: Effect of birth rate r_{birth} on population evolution.

4 Implementation details

4.1 Overview

This project implements the Penna model of biological aging in C++17. The core abstractions are the `Genome` class, which encodes an individual’s bit-string genome, the `Animal` class, which wraps a genome together with age and mutation state, and the `Population` class, which collects animals in a vector and provides access to population-wide functionalities such as aging, reproduction, and death, and access to quantities such as the average age.

4.2 Build and Project Structure

The project is organized with sibling `source_dir` and `build_dir` directories under a common parent, and is built using CMake targeting the C++17 standard. Compiler warnings are enabled via `-Wall -Wextra -Wpedantic` to surface common pitfalls early.

4.3 Domain Representation

Genome bits are stored in `std::bitset<B>`, with the bit-width `B` fixed at compile time (typically 64). This container was chosen over `std::vector<bool>` and `boost::dynamic_bitset` for performance reasons.

Simulation parameters – maximum age (`maxage`), mutation rate (`M_`), mutation threshold (`T_`), and reproductive age (`R_`) – are shared across all instances as `static` class members.

Random number generation uses `std::mt19937`, seeded once with a fixed seed. The RNG engine and seed are held in an anonymous namespace within `genome.cpp`, with an `init_rng()` function declared in the header and invoked once from `main()`.

4.4 Key Design Decisions

- **genome_size as static constexpr:** chosen over `static const` to avoid Object Definition Rule (ODR) issues that arise with `static const` integral members defined in headers.
- **Static member definitions:** `M_`, `maxage_`, `T_`, and `R_` are defined in their corresponding `.cpp` files.
- **Bounds-checked bit access:** `gene_at(i)` uses `.test(i)` rather than `operator[]` to guarantee bounds checking.
- **Age-genome invariant:** `maxage_` is capped at `genome_size - 1` to prevent out-of-bounds bitset access.
- **Constructors:** `Animal` provides both a default constructor (all-false genome) and a constructor taking a `const Genome&`.
- **Parameter validation:** invalid simulation parameters are handled by clamping to valid ranges with a warning written to `std::cerr`, chosen as a pragmatic middle ground between `assert` and thrown exceptions.
- **RNG state scoping:** the RNG engine and seed are kept in an anonymous namespace (translation-unit-local linkage) rather than at global scope, which is the idiomatic way to encapsulate such state without exposing external linkage.

References

- [1] T. J. P. Penna. A bit-string model for biological aging. *J. Stat. Phys.*, 78(1629), 1995.
- [2] S. Moss de Oliveira, T.J.P. Penna, and D. Stauffer. Simulating the vanishing of northern cod fish. *Physica A: Statistical Mechanics and its Applications*, 215(3):298–304, 1995.